

Homework 03

Date: _____

Syed Aham Hussain

SH10331

Computer Architecture

Question 1)

1) The clock cycle time for pipelined version depends upon the longest path delay and buffer register $\therefore 320\text{ps} + 25\text{ps}$
 $\therefore 345\text{ps}$

For a non pipelined version $\therefore 220\text{ps} + 200\text{ps} + 320\text{ps} + 260\text{ps} + 180\text{ps} = 1180\text{ps}$

2) As latency is the time taken by an instruction from start to finish. Store utilizes all the 5 stages, so for non pipelined version: 1 clock $\times 1180\text{ps} = 1180\text{ps}$.

In a pipelined version: 5 stages $\times 345\text{ps}$
 $\therefore \text{MT} \therefore 1725\text{ps}$.

$\therefore$ ~~latency~~ Pipelining does not make a single instruction faster.

Speedup $\therefore \frac{\text{Non pipelined time}}{\text{Pipelined time}}$

2) Non Pipelined time $\Rightarrow 1200 \times 1180 \Rightarrow 1416000 \text{ ps}$

2) Pipelined time $\Rightarrow (1200 + 5 - 1) \times 345 \text{ ps}$

$\Rightarrow 1204 \times 345 \Rightarrow 415380 \text{ ps}$

Speed up $\Rightarrow \frac{1416000}{415380} \approx 3.41$

$\therefore$ The maximum speedup that could be achieved is in stages = 5.

4) Now as a stall is occurring every 5 instructions.

Stall cycles $\Rightarrow \frac{1200}{5} \times 5 \Rightarrow 240$

Total pipelined cycles $\Rightarrow 240 + 1200 \Rightarrow 1440 \text{ cycles}$

Pipelined time $\Rightarrow 1440 \times 345 \Rightarrow 496800 \text{ ps}$

Speedup $\Rightarrow \frac{1416000}{496800} \approx 2.84$

$\therefore$ stall made speed up decreased

1.5) Our pipelined time [longest-time taken in data path] is for EX = 320ps.

We can split this stage into 2 stages which won't have an affect on ISA.

And [EX1 and EX2] combined will now make

total no of stages $(m) = C$, which would increase the maximum speedup that could be achieved $\approx C$, reducing cycle time at cost of more stages and hazard stall.

Now the latency becomes next worse:

$$\text{New clock} \approx 260 + 25 = 285 \text{ ps}$$

↑
MEM

This is called pipeline deepening.

G) Instruction memory will be used in Instruction Fetching (IF) stage by every instruction.

So utilization is 100%.

Data memory is used in MEM stage only in load and store instruction from instruction mem:

load $\approx 25\%$

store $\approx 25\%$

total $\approx 50\%$ utilization.

Question 02)

1) For a non pipelined version:

We reduced the no. of stages as EX+MEM constitute a single stage now;

In original ISA, longest instruction was load because it used all 5 stages @ 1180ps.

The ~~EX~~ ~~now~~ ~~would~~ ~~take~~ ~~time~~ for

EX+MEM = 320 ps as EX is a bigger.

New single clock cycle = IF + ID + EX + WB

$$= 200 + 220 + 320 + 180$$

$$= 920 \text{ ps.}$$

2) Our new CPU is faster per cycle with 920ps but every ld and sd becomes 2 instructions. For instruction mix:

ld + sd = 50% which become double

Originally, we had 1200 instructions, 600 of these are ld and sd. This means that 600 more will get added.

Total instruction count = 1200 + 600 = 1800.

$$\rightarrow \text{Old time} = 1200 \times 1180 \text{ ps} = 1416000 \text{ ps}$$

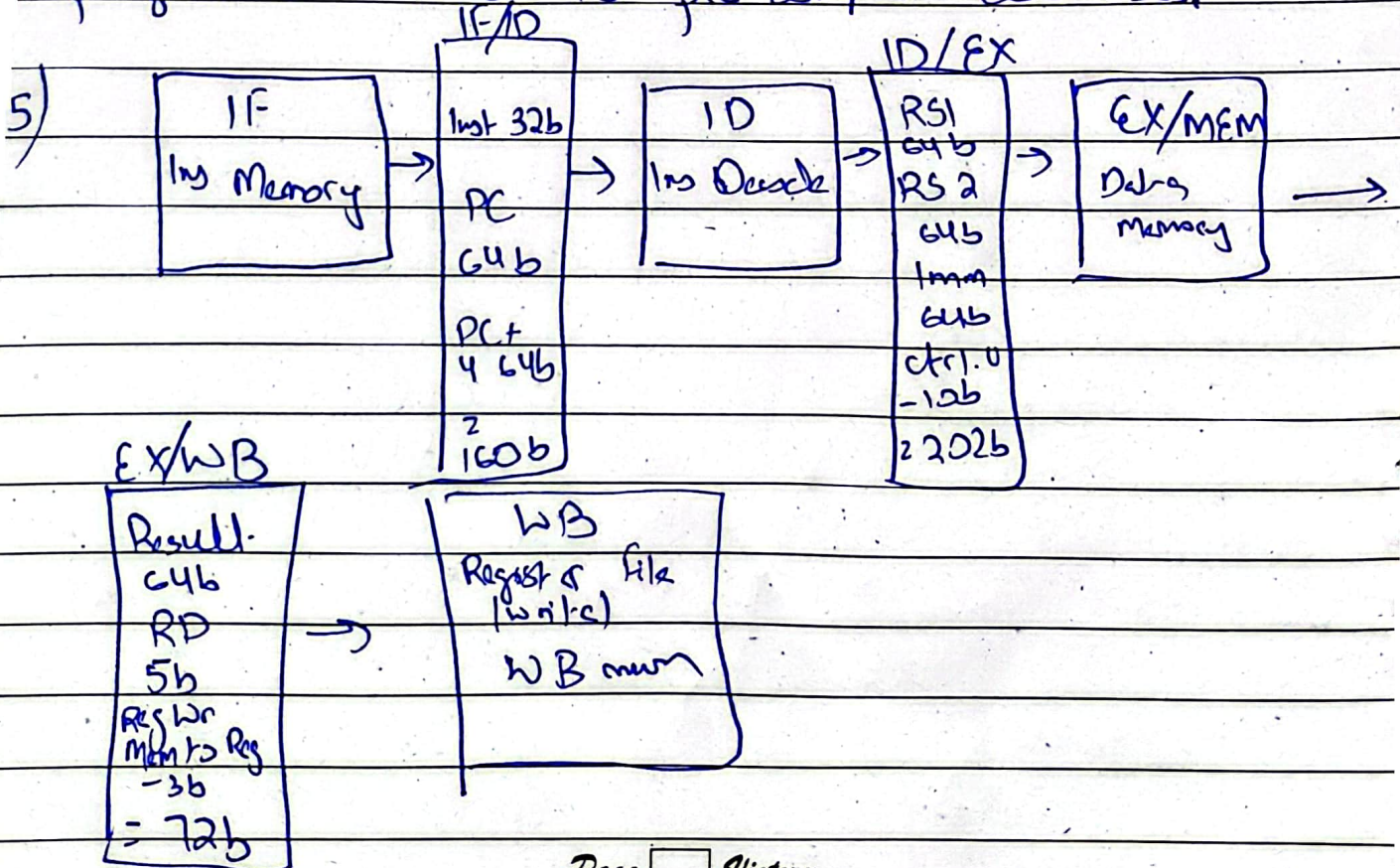
$$\rightarrow \text{New time} = 1800 \times 920 = 1656000 \text{ ps}$$

The new time > old time, making our CPU slower by $\frac{1656000}{1416000} = 1.17$ times.

3) The factor that would make program run faster or slower on New CPU is no of ld and sd instructions in program. If it has fewer ld/sd, the IC is less and shorter cycle time will make new CPU faster. But in a case like our ~~ISA~~ instruction mn (50%) of ld/sd, high IC outweighs the cycle time savings.

4) The original CPU is better;

- ~~better memory management~~ and less blow up in memory intensive programs.
- New ISA make programming difficult as programmer needs to pre compute addresses.



The EX and MEM stage got merged into one combined stage.

The new pipeline is ~~IF~~ IF $\rightarrow$ ID $\rightarrow$ EX/MEM $\rightarrow$ WB.
 The effective latency of this stage is $\max(\text{EX}, \text{MEM})$
 $\Rightarrow \max(320, 260)$
 $\Rightarrow 320 \text{ ps}$.

New pipelined clock = $320 + 25 = 345 \text{ ps}$.

$\Rightarrow$ Interstage register sizes:

IF/ID: instruction (32 bits) + PC (64 bits) = 96 bits

ID/EX: Two reg (64 + 64) + imm (64) + control signal (10 bits) = 206.

EX/WB: ALU result (64 bits) + rd (5) + write ctrl.

6) Fewer stages means fewer inter stage registers. With 4 stages, only 3 inter stage registers.

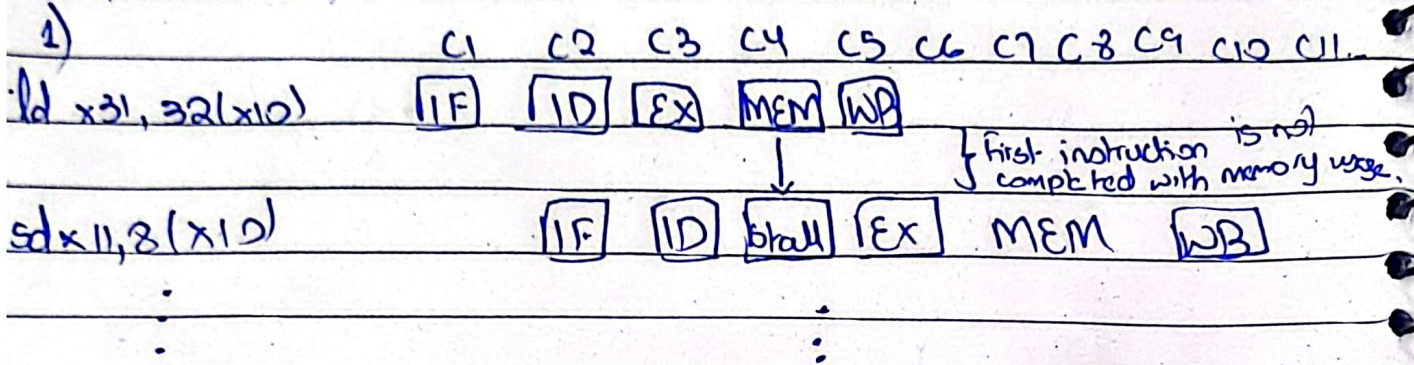
Also, for the combined stage the cycle time stays same (345 ps) since EX was already bottleneck.

7) Fewer stages means fewer ^{hazard} data stalls ~~and~~ b/w dependent instructions.

As the hazards would cause fewer wasted cycles.

8) The instruction count ~~almost~~ doubles for ld/st - heavy programs increasing no of cycles needed. Also the compiler has to do extra work computing addresses.

Question 03)



MEM = memory access cycle.

stall = IF stage's blocked because memory is busy in MEM stage.

The conflict happens at cycle 4:

(ld)11 is in the MEM stage at cycle 4 - It is in need of memory for data read.

(sd)12 wants to IF in C4. It requires memory too.

Same memory,
Same cycle

Structural
Hazard

2) Reordering can help with data hazards (by moving independent instructions in b/w), but the structural hazard occurs at fixed pipeline position regardless of order.

∴ Everytime instruction reaches MEM stage, the next instruction fetch is blocked

3) Yes, we could insert NOP b/w every memory accessing (ld/st) and next instruction. So that when memory instruction is in MEM stage, the NOP occupies IF and no instruction is trying to use memory at same time.

However, the cycle gets wasted, costing performance and time. And hardware is doing this automatically to prevent hazard. So manually inserting NOP's won't give any benefit.

4) A stall would occur 50% of all ~~the time~~ instructions. Every 2 instruction, we would have a stall. For 1000 instructions $\Rightarrow$ 500 stall cycles.

Question 04)

1) RegWrite is normally 0 for sd and beq, because they don't require writing to registers.

If this is stuck at 1 that would mean every instruction is writing to register

sd would be ~~writing~~ ^{whatever is on} a garbage value to destination register. The beq instruction the 1 write back ~~is~~ gets written on to register corrupting it.

∴ Every instruction becomes register writing instruction.

2) Only beq requires MemRead. If stuck at 1, every instruction would read from memory.

The address in memory will be whatever ALU computed.

For R-type instructions its arithmetic result, not a valid mem address.

This causes spurious memory read at every instruction. While it is not writing anything corrupting registers. It is wasting memory bandwidth and can cause faults.

3) MemWrite, only sd asserts it.

Every instruction writes to memory using whatever address ALU computed and whatever RS2 holds. Memory gets corrupted at every single instruction.

4) BranchZ1 enables AND gate that computes $PCSrc \wedge Branch \wedge Zero$. If stuck at 1, every instruction that produces ZeroZ1 will trigger branch.

Example: Add instruction that produces a resultZ0 will cause PC to jump at wrong address. Non zero results are unaffected.

5) $ALUSrc = 1$ only for ld, sd, addi (I-type).

Using sign extended imm as ALU's second input ~~output~~. For R-type $ALUSrc = 0$, means use R2 not immgen. In this case, all R-type will use imm generator for R2 producing wrong results (add, sub, and, or, etc).

6) $PCSrc$ selects b/w $[PC+4]$ normal execution and branch target execution. When always at 1, it would send the PC to wrong branch addresses ~~selecting~~ affecting the flow of program. Gives wrong results for non branch instructions. Processor may crash.

7) This causes ALU control to look at func 3/func 7 field to decide operation instead of forcing ADD. The address calculation becomes wrong (AND, OR, XOR etc), ld/sd compute wrong memory locations.

8) For ld/sd, ALUop should be 00, and for R-type it should be 10. If ALUop0 is 1. ld/sd get ALUop = 01 [subtract], so address calculation becomes subtraction, wrong addresses, wrong memory access. R-type instructions get ALUop = 11 which is undefined behaviour in ALU control logic, producing wrong results.

Question 05)

Instructions

Hazards with previous ins.

11: add x1, x0, 5	None
12: addi x2, x0, 10	read only x0
13: add x3, x1, x2	RAW on x1 (11) - EX/EX forwardable
14: sub x4, x3, x1	RAW on x3 & (13) - EX/EX forwardable
15: lw x5, 0, (x4)	RAW on x4 (14) - EX/EX forwardable lw causes load use hazard for 16.
16: add x6, x5, x1	RAW on x5 (from 15 lw) - LOAD use hazard. 1 stall + MEM/EX forward.

17: sw x6, 4(x4) RAW on x6(16) - EX/EX forwardable
 RAW on x4(14) - MEM/EX forwardable
 18: add x7, x6, x3 RAW on x6(16) - MEM/EX forwardable
 RAW x3(13) - already in Reg file.

1) All hazards are RAW (Read after write).
 13 reads x1 (written by 11, 2 apart) ...

2) Instruction	Expected	Actual.
11: add x1, x0, 5	x1 = 5	x1 = 5 (correct)
12: add x2, x0, 10	x2 = 10	x2 = 10 x1=0, x2=0
13: add x3, x1, x2	x3 = 15	x3 = 0 (reads stale)
14: sub x4, x3, x2	x4 = 10	x4 = 0 (stale)
15: lw x5, 0(x4)	x5 = Mem[0] = 40	x5 = Mem[0] (wrong address)
16: add x6, x5, x1	x6 = 45	x6 = 0 (stale)
17: sw x6, 4(x4)	Mem[14] = 45	Mem[4] = 0 (stale)
18: add x7, x6, x3	x7 = 60	x7 = 0

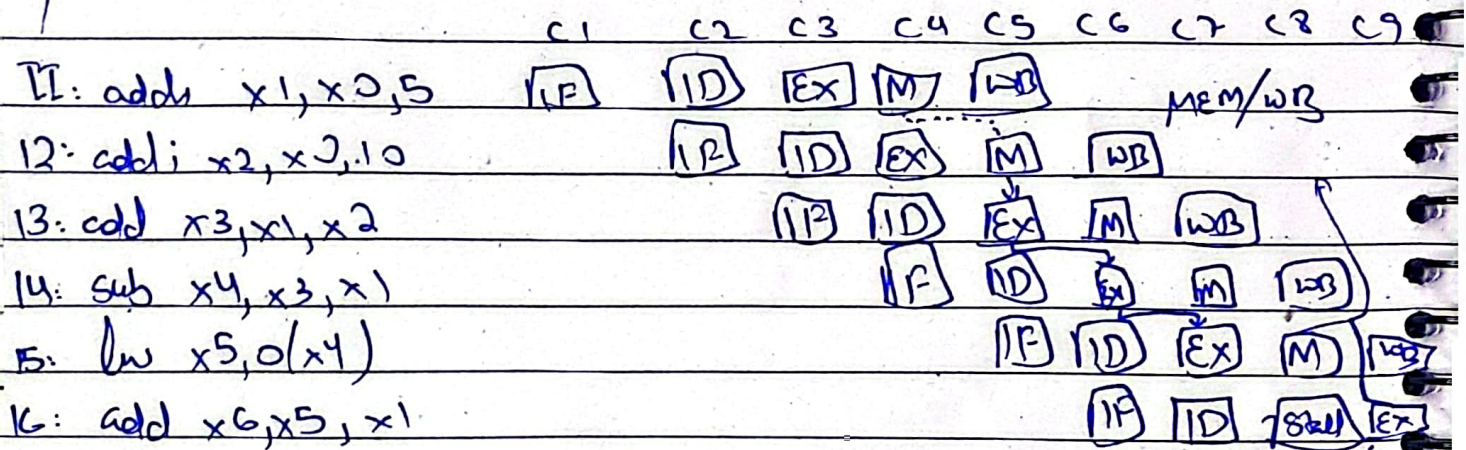
Without forwarding, every instruction is read at ID stage, where previous instruction is not completed.

3) Forwarding resolves most hazards by routing results directly from the pipeline register to ALU input without waiting for WB.

There are 2 forwarding paths. EX/MEM forward and MEM/WB forward.

The one hazard that cannot be fixed by forwarding is R→R (read use hazard). It doesn't have data until end of MEM stage, IC needs it at start of EX. There is no other way to get data in time without 1 stall cycle. The hazard detection unit detects this, freezes IC in ID for one cycle then the MEM/WB forward delivers correct value correctly.

4)



17:



18:



∴ There is exactly 2 stall in entire program, b/w 15 and 16. All others are resolved by forwarding. The stall is unavoidable because the V based value from lw doesn't exist until end of MEM stage.

Question 06)